模擬問題3

Ｐｙｔｈｏｎプログラミング能力認定試験

１　級

解答時における注意事項

1．次の表に従って解答してください。

問題番号	問１～問７
選択方法	７問必須
試験時間	９０分

2．HB の黒鉛筆を使用してください。訂正する場合は，あとが残らないように消しゴムできれいに消し，消しくずを残さないでください。なお，ボールペンや万年筆等で記入した場合は，採点されません。

3．マークシート（解答用紙）の所定の欄に，級種，会場コード，受験番号を記入しマークしてください。また，会場名，氏名及びフリガナを所定の位置に記入してください。

4．解答は，次の例題にならって，「解答マーク欄」にマークしてください。

〔例題〕　日本の首都はどこか。

ア　東京　　イ　京都　　ウ　大阪　　エ　福岡

正しい答えは“ア　東京”ですから，次のようにマークしてください。

例題　■　イ　ウ　エ

指示があるまで開いてはいけません。 試験終了後，問題冊子を回収します。

受験会場	
受験番号	
氏　　名	

サーティファイ
情報処理能力認定委員会

問1～問7は，すべて必須問題です。全問について解答してください。
各設問の答えは，解答群の中から一つだけ選び，括弧中の設問番号に対応したマークシートの解答番号の「解答マーク欄」にマークしてください。なお，二つ以上マークした場合には不正解になります。

問1　Pythonの特徴やオブジェクト指向プログラミングに関する次の記述の正誤を，解答群の中から選べ。

(1)　すべての変数に型を明示的に指定する必要がある。

(2)　del文を使用することで，タプル 型から要素を削除することができる。

(3)　関数をオブジェクトとして扱うことができる。

(4)　関数の引数に「*args」を指定し，引数に何も指定されなかった場合，エラーになる。

(5)　クラス内に定義されたインスタンス変数は，インスタンスごとに異なる値を持つことができる。

(6)　クラスの外部からクラスメソッドを呼び出すとき，「クラス名.メソッド名()」と記述する。

解答群

ア　正しい　　イ　誤り

問2　次のプログラムおよびクラスやオブジェクトに関する説明を読んで，［　　］に入れる適切な字句を，解答群の中から選べ。

＜プログラム＞

```
class Gadget:
    _category = 'Device'

    def __init__(self, name, price):
        self.name = name
        self.price = price

    @classmethod
    def get_category(cls):
        return cls._category

    @staticmethod
    def is_expensive(price):
        return price > 1000

    def update_attribute(self, attr_name, value):
        if hasattr(self, attr_name):
            setattr(self, attr_name, value)
        else:
            print(f"属性'{attr_name}'は存在しません。")

    def __str__(self):
        return f'{self.name} (${self.price})'

class AdvancedGadget(Gadget):
    _category = 'Advanced Device'

    def __init__(self, name, price, features):
        super().__init__(name, price)
        self.features = features

    def __str__(self):
        return f"{super().__str__()} : {', '.join(self.features)}"
```

＜クラスやオブジェクトに関する説明＞

1. Gadget クラスの_category は [(7)] である。_category を [(7)] として定義する目的は，[(8)] である。
2. 9 行目で引数に設定されている cls は，[(9)] を参照している。
3. is_expensive メソッドは，[(10)] メソッドであり，これを [(11)] という。
4. 18 行目の setattr は，オブジェクトの属性を [(12)] できる関数である。
5. AdvancedGadget クラスは Gadget クラスを [(13)] している。また，28 行目から定義している__init__メソッドは，[(14)] 処理を行う。

(7)の解答群

ア　インスタンス変数
イ　クラス変数
ウ　グローバル変数
エ　ローカル変数

(8)の解答群

ア　値の変更を完全に禁止するため
イ　変数のスコープを変更するため
ウ　クラス外部の関数からアクセスしやすくするため
エ　クラス全体で共通の値を管理するため

(9)の解答群

ア　インスタンス
イ　クラス自体
ウ　クラス変数
エ　クラスメソッド

(10)の解答群

ア　インスタンス変数を操作する
イ　インスタンスの初期化を補助する
ウ　インスタンスやクラスに直接依存せず，引数だけで動作する
エ　クラス変数を操作する

(11)の解答群

ア　クラスメソッド
イ　コンストラクタ
ウ　ジェネレータ
エ　スタティックメソッド

(12)の解答群

ア　削除
イ　初期化
ウ　制限
エ　動的に変更

(13)の解答群

ア　インスタンス化
イ　オーバーライド
ウ　拡張
エ　継承

(14)の解答群

ア　親クラスのコンストラクタを拡張し，新しい変数 features を追加する
イ　親クラスのコンストラクタを無効化し，新しい変数 name, price, features を追加する
ウ　変数 name, price, features をグローバル変数として定義する
エ　変数 features をグローバル変数として定義する

問3　次のプログラムに対する設問の答えとして適切なものを，解答群の中から選べ。

(15) 次のプログラムを実行した結果，「AustriaC!n!d!」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   list_a = ['Austria', 'Brazil', 'Canada', 'Denmark']
2   text = ''
3
4   for i in list_a:
5       if i.startswith('A'):
6           text += i
7       elif [          ]:
8           i = i.replace('a', '!')
9           text += i
10
11  print(text)
```

解答群

ア　i.count('a') <= 2
イ　i.count('a') >= 2
ウ　i.find('a') <= 2
エ　i.find('a') >= 2

(16) 次のプログラムを実行した結果，「[2, 1]」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   dict_a = {'waterfall':1, 'snowfall':2, 'watersnow':3, 'fallsnow':4}
2   dict_b = {}
3
4   for name in sorted(dict_a):
5       i = name[          ]
6       if i not in dict_b:
7           dict_b[i] = []
8       dict_b[i].append(dict_a[name])
9
10  result = dict_b.get('fall', '該当なし')
11  print(result)
```

解答群

ア　:-4　　イ　:4　　ウ　-4:　　エ　4:

(17) 次のプログラムを実行した結果，「{'A': 35, 'B': 25, 'C': 40}」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   list_a = [('A', 10), ('B', 20), ('C', 30), ('A', 15), ('B', -5)]
2   numbers = {}
3
4   for player, num in list_a:
5       match numbers.get(player):
6           case [　　]:
7               numbers[player] = num + 10
8           case current_num:
9               numbers[player] = current_num + num
10
11  print(numbers)
```

解答群

ア　case 0
イ　case False
ウ　case None
エ　if numbers[player] is None

(18) 次のプログラムを実行した結果，「[1, 2, 3, 4, 5, 6]」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   list_a = [[1, 2], [3, 4], [5, 6]]
2   list_b = [　　]
3
4   print(list_b)
```

解答群

ア　x for sublist in list_a
イ　x for sublist in list_a for x in sublist
ウ　x for x in sublist
エ　x for x in sublist for sublist in list_a

(19) 次のプログラムを実行した結果，「[20, None, 6, 3]」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   tuple_a = (0, 10)
2
3   def calculate(value):
4       if value > 0:
5           return value * 2
6       elif value in tuple_a:
7           [　　]
8       else:
9           return value + 5
10
11  print([calculate(v) for v in (10, 0, 3, -2)])
```

解答群

ア　continue

イ　pass

ウ　return 0

エ　yield

問4　次のプログラムに対する設問の答えとして適切なものを，解答群の中から選べ。

(20) 次のプログラムを実行した結果として，正しいものはどれか。

```
1   x = 0
2   list_a = ['Yamada', 'Aoki', 'Nakamura', 'Murakami']
3
4   for y in range(0, 5):
5       a, b = divmod(y, 2)
6       if a == 0:
7           x -= 1
8       else:
9           x += b
10          continue
11
12  print(x, list_a[x])
```

解答群

ア　「-1 Murakami」が表示される。
イ　「0 Yamada」が表示される。
ウ　「1 Aoki」が表示される。
エ　「2 Nakamura」が表示される。

(21) 次のプログラムを実行した結果として，正しいものはどれか。ただし，要素内のデータは順不同のものとする。

```
1   list_a = ['C', 'A', 'D', 'B', 'A', 'B']
2   list_b = [1, 2, 3, 4, 5]
3
4   set_a = set(list_a)
5   result = sorted(zip(set_a, list_b))
6   print(result)
```

解答群

ア　「[('A', 1), ('B', 2), ('C', 3)]」が表示される。
イ　「[('A', 1), ('B', 2), ('C', 3), ('D', 4), ('A', 5)]」が表示される。
ウ　「[('A', 3), ('B', 4), ('C', 2), ('D', 1)]」が表示される。
エ　5行目でエラーになる。

(22) 次のプログラムを実行した結果として，正しいものはどれか。

```
1   def calculate(list_a):
2       i = int(sum(list_a) / len(list_a))
3       for j in list_a:
4           if i < j:
5               i += j
6           else:
7               i -= j
8       return i
9
10  n = calculate([1, 2, 3, 4, 5])
11  print(n)
```

解答群

ア　「0」が表示される。
イ　「2」が表示される。
ウ　「6」が表示される。
エ　「18」が表示される。

(23) 次のプログラムを実行した結果として，正しいものはどれか。

```
1   x = 0b111111
2   y = 0b101010
3   z = x >> 3
4
5   a = x & y
6   b = x ^ z
7   c = a | b
8
9   print(bin(c))
```

解答群

ア　「0b100101」が表示される。
イ　「0b110000」が表示される。
ウ　「0b111010」が表示される。
エ　「0b111110」が表示される。

(24) 次のプログラムを実行した結果として，正しいものはどれか。

```
1  value = 0.123456
2  for i in range(1, 5):
3      rounded = round(value, i)
4      if rounded > value:
5          break
6
7  print(rounded, i)
```

解答群

ア 「0.1234 4」が表示される。
イ 「0.12345 5」が表示される。
ウ 「0.1235 4」が表示される。
エ 「0.1235 5」が表示される。

問５　次のプログラムを実行したとき，print関数によって表示される値を，解答群の中から選べ。

＜プログラム＞

```
1   list_a = [1, 2, 3, 4, 5]
2
3   result = sum(list_a) > 15
4   print(result)                                          … (25)
5
6   list_b = list_a.copy()
7   num = len(list_b) // 2
8   for i in range(num):
9       list_b[i], list_b[num + i] = list_b[num + i], list_b[i]
10  print(list_b)                                          … (26)
11
12  num_list = [x for x in list_a if x % 2 == 0]
13  ratio = len(num_list) / len(list_a) * 100
14  print(f'{ratio:.2f}%')                                … (27)
15
16  list_c = list(reversed(list_a))
17  list_d = []
18  n = 0
19  while n < len(list_a):
20      if 1 <= abs(list_a[n] - list_c[n]) <= 3:
21          list_d.append((list_a[n], list_c[n]))
22      n += 1
23  print(list_d)                                          … (28)
24
25  list_e = [5, 10, 15, 20, 25]
26  zip_a = zip(list_a, list_e)
27  result = list(filter(lambda x: x[1] == x[0] ** 2, zip_a))
28  print(result)                                          … (29)
```

(25) の解答群

ア 15　　イ False　　ウ None　　エ True

(26) の解答群

ア [1, 3, 5, 4, 2]
イ [3, 4, 1, 2, 5]
ウ [3, 4, 5, 1, 2]
エ [4, 5, 1, 2, 3]

(27) の解答群

ア 40.00%
イ 40%
ウ 60.00%
エ 60%

(28) の解答群

ア [(1, 5), (2, 4), (3, 3), (4, 2), (5, 1)]
イ [(1, 5), (2, 4), (4, 2), (5, 1)]
ウ [(2, 4), (3, 3), (4, 2)]
エ [(2, 4), (4, 2)]

(29) の解答群

ア [(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
イ [(1, 5), (4, 20)]
ウ [(3, 9), (5, 25)]
エ [(5, 25)]

問6　次のプログラムの説明およびプログラムを読んで，プログラム中の［　　］に入れる適切な字句を，解答群の中から選べ。

＜プログラムの説明＞

所有する電化製品と各電化製品の消費電力が記録された appliances.json を読み込み，一覧化した節電のアクションを選択し，全体の消費電力を目標値まで削減する節電シミュレーションを行うプログラムである。本プログラムでは，元の消費電力の 30%削減を目標として設定する。appliances.json のデータ構成およびデータは後述する。

プログラムは，以下のような流れで実行される。この流れの例を，図1に示す。

① 所有する電化製品に合わせてプログラムから節電の提案が複数出される。
② 提案の中からユーザーは好みのアクション（1～6）を選択する。
③ 全体の消費電力が目標値以下となるまで，②を繰り返す。
④ 最終的に，選択したアクションをすべて表示する。

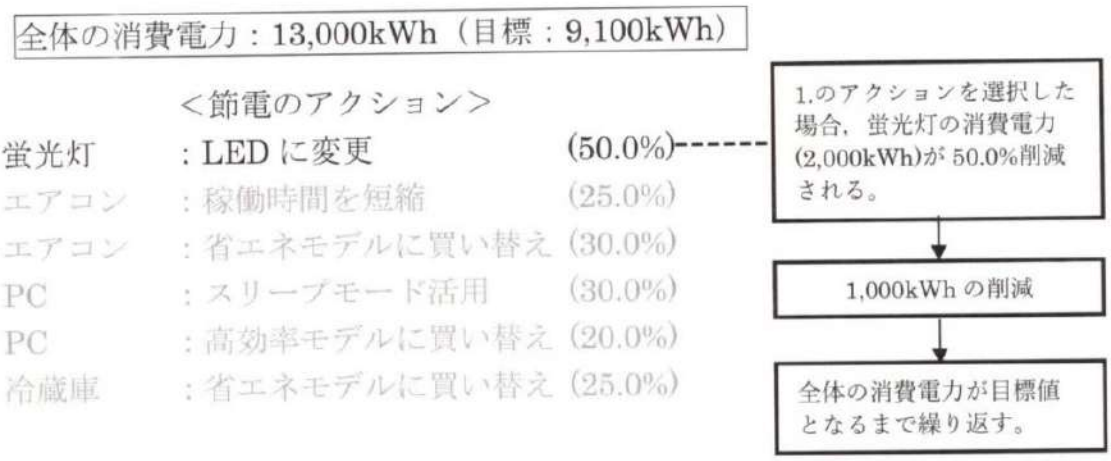


図1　実行の流れの例

＜処理手順＞

(1) appliances.jsonに記録されているデータを読み込み，変数appliancesに格納する。
(2) appliancesから消費電力を読み込み，全体の消費電力を計算した値を変数totalにtotalを30%削減した値を変数targetにそれぞれ格納する。
(3) 変数actionsに格納されたアクションを番号付きで表示する。
(4) ユーザーにアクションの選択を促し，選択されたアクションの番号を変数choiceに格納する。なお，choiceは整数以外の入力はないものとする。choiceが0以下か，もしくはactionsの要素の数より大きい場合，「無効な選択です。番号を確認してください。」と表示し，再度入力を促す。
(5) 変数actionにユーザーが選択したアクションを格納する。もし，action['executed']がTrueである場合は「このアクションはすでに実行済みです。」と表示し，(4)の処理に戻る。
(6) 選択したアクション通り，消費電力を計算する。まず，選択した電化製品の消費電力と，選択したアクションの削減率を乗算した値を変数reductionに格納する。ここで，

reductionは，選択したアクションによって削減される消費電力を表す。次に，各電化製品の消費電力は選択したアクションによって削減された値を保持するため，アクションによって削減された電化製品の消費電力からreductionを差し引く。また，totalからもreductionを差し引き，現在の消費電力を計算する。最後に，選択したアクションを示す文字列をリストtakenに追加し，action['executed']をTrueに置き換える。

(7) totalがtargetより大きいとき，(4)～(6)の処理を繰り返す。

(8) takenと最終的な消費電力を表示する。

＜データの構成と記録されているデータの例＞

appliances.jsonのデータの構成を，図２に示す。

appliances.json
[　{"machine": "電化製品名", "power": 消費電力}, 　{"machine": "電化製品名", "power": 消費電力}, 　⋮]

図２　appliances.jsonのデータの構成

appliances.json に記録されているデータの例を，図３に示す。

appliances.json
[　{"machine": "蛍光灯", "power": 2000}, 　{"machine": "エアコン", "power": 6000}, 　{"machine": "PC", "power": 3000}, 　{"machine": "冷蔵庫", "power": 2000}]

図３　appliances.json に記録されているデータの例

このプログラムで使用するライブラリと関数の説明を，次に示す。

json ライブラリ
load 関数
load(fp, 省略) JSON ファイルを読み込み，読み込んだデータを返す。 引　数：fp　JSON ファイルのファイルオブジェクト 戻り値：読み込んだデータ

≪実行結果例≫

```
=== 選択可能なアクション ===
1.蛍光灯:LEDに変更 (50.0%)
2.エアコン:稼働時間を短縮 (25.0%)
3.エアコン:省エネモデルに買い替え (30.0%)
4.PC:スリープモード活用 (30.0%)
5.PC:高効率モデルに買い替え (20.0%)
6.冷蔵庫:省エネモデルに買い替え (40.0%)

現在の消費電力:13000.0kWh (目標:9100.0kWh)
実行するアクションの番号を選択: 1

現在の消費電力:12000.0kWh (目標:9100.0kWh)
実行するアクションの番号を選択: 6

現在の消費電力:11200.0kWh (目標:9100.0kWh)
実行するアクションの番号を選択: 3

現在の消費電力:9400.0kWh (目標:9100.0kWh)
実行するアクションの番号を選択: 3
このアクションはすでに実行済みです。

現在の消費電力:9400.0kWh (目標:9100.0kWh)
実行するアクションの番号を選択: 2

=== 節電シミュレーション結果 ===
蛍光灯:LEDに変更
冷蔵庫:省エネモデルに買い替え
エアコン:省エネモデルに買い替え
エアコン:稼働時間を短縮
最終消費電力: 8350.0 kWh  30%削減を達成しました！
```

<プログラム>

```
import json

file_name = 'appliances.json'
    (30)    (file_name, 'r', encoding = 'utf-8') as fp:
    appliances = json.load(fp)

total = sum(x['power'] for x in appliances)
target = total * 0.7

actions = [
    {'machine': '蛍光灯', 'action_name': 'LED に変更',
     'rate': 0.5, 'executed': False},
    {'machine': 'エアコン', 'action_name': '稼働時間を短縮',
     'rate': 0.25, 'executed': False},
    {'machine': 'エアコン', 'action_name': '省エネモデルに買い替え',
     'rate': 0.3, 'executed': False},
    {'machine': 'PC', 'action_name': 'スリープモード活用',
     'rate': 0.3, 'executed': False},
    {'machine': 'PC', 'action_name': '高効率モデルに買い替え',
     'rate': 0.2, 'executed': False},
    {'machine': '冷蔵庫', 'action_name': '省エネモデルに買い替え',
     'rate': 0.4, 'executed': False},
]

print('=== 選択可能なアクション ===')
for i, j in enumerate(actions):
    print(f"{i+1}.{j['machine']}:{j['action_name']}", end='')
    print(f"({j['rate']*100:.1f}%)")

taken = []
while   (31)   :
    print(f'¥n 現在の消費電力:{total:.1f}kWh (目標:{target:.1f}kWh)')
    choice = int(input('実行するアクションの番号を選択: ')) - 1

    if 0 <= choice < len(actions):
        action = actions[choice]
        if   (32)   :
            print('このアクションはすでに実行済みです。')
            continue

        for appliance in appliances:
            if appliance['machine'] == action['machine']:
                reduction =   (33)
                appliance['power'] -= reduction
                total -= reduction
```

```
46                      (34)(f"{action['machine']}:{action['action_name']}")
47                      action['executed'] = True
48                      break
49          else:
50              print('無効な選択です。番号を確認してください。')
51
52      print('¥n=== 節電シミュレーション結果 ===')
53      print('¥n'.join(taken))
54      print(f'最終消費電力: {total:.1f} kWh  30%削減を達成しました！')
```

(30) の解答群

ア　fp = open
イ　open
ウ　read open
エ　with open

(31) の解答群

ア　total < target
イ　total > target
ウ　total == target
エ　total != target

(32) の解答群

ア　action['executed']
イ　action['executed'] is 1
ウ　not action['executed']
エ　True

(33) の解答群

ア　action ['power'] * appliance['rate']
イ　action ['power'] * appliance['rate'] / 100
ウ　appliance['power'] * action['rate']
エ　appliance['power'] * action['rate'] / 100

(34) の解答群

ア　print
イ　taken += int
ウ　taken.append
エ　taken.join

問7　次のプログラムの説明およびプログラムを読んで，プログラム中の［　　］に入れる適切な字句を，解答群の中から選べ。

＜プログラムの説明＞

顧客の年間購入金額と購入頻度が記録された customer_data.csv を読み込み，顧客を三つのクラスター（グループ）に分類してクラスター分析を行い，分析結果をグラフとして表示するプログラムである。customer_data.csv のデータ構成およびデータは後述する。

クラスター分析は，個々のデータから似ているデータ同士を集めて，いくつかのクラスターに分類する分析手法である。本プログラムでは，ユークリッド距離（データ点間の直線距離）が近いものでクラスターを分類する。

本プログラムにおけるクラスター分析は，次の手順で行う。

○クラスター分析の手順

① 各データを一つのクラスターとして扱う。つまり，customer_data.csv は 100 件のデータをもつので，100 個の独立したクラスターとして扱う。

② 各データ間のユークリッド距離を求め，行列 D に格納する。ここで，データ点 (x_1, y_1) と (x_2, y_2) の間のユークリッド距離は，次の式で求められる。

$$\mathrm{D}(1,2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

③ 行列 D を参照し，最も距離の小さい二つのクラスターを特定し，クラスター同士を統合する。統合したクラスターは一つのクラスターとみなす。

④ 統合したクラスターの代表点（重心）を求める。代表点は，クラスターに含まれるデータの平均値で求められる。ここまでのクラスター分析の例を，図1に示す。

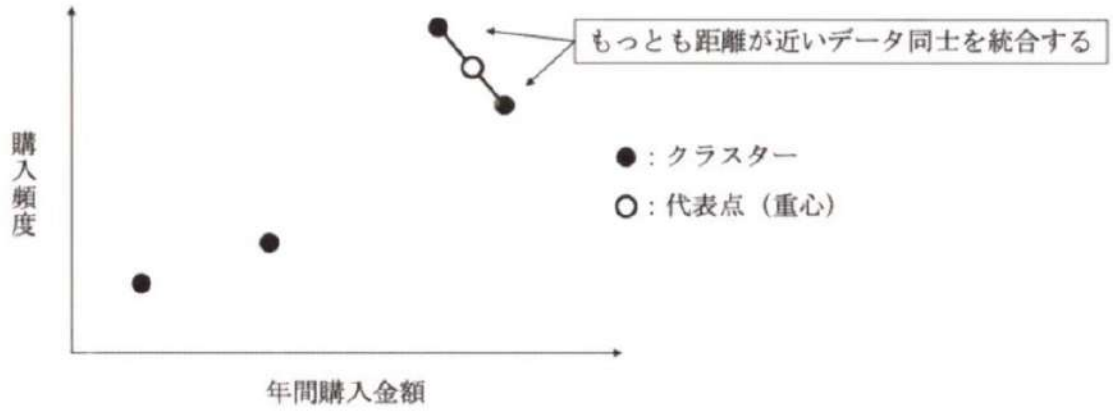


図1　クラスター分析の例

⑤ 代表点をクラスターの基準点とし，各クラスター間のユークリッド距離を再計算する。

⑥ クラスターの数が 3 になるまで，③～⑤の手順を繰り返す。

<データの構成と記録されているデータの例>

customer_data.csvのデータの構成を，図2に示す。

customer_data.csv
年間購入金額,購入頻度 年間購入金額1,購入頻度1 年間購入金額2,購入頻度2 ⋮

図2　customer_data.csvのデータの構成

customer_data.csv に記録されているデータの例を，図3に示す。なお，年間購入金額と購入頻度は，スケールの違いを考慮し，いずれも小数点以下 2 桁で正規化（0〜1 の範囲）されている。

customer_data.csv
年間購入金額,購入頻度 0.09,0.01 0.06,0.04 0.09,0.04 0.12,0.03 0.06,0.04 ⋮

図3　customer_data.csv に記録されているデータの例

<クラスの仕様>

○ClusterAnalysis クラス

データを三つのクラスターに分類し，その結果をグラフで表示する。

- __init__ メソッド

  インスタンスの初期化を行う。

- fit メソッド

  データのクラスター分析を行う。

  ・処理手順

  (1)　クラスターの結合状況を示す変数 clusters を初期化する。 distance_matrix 関数を使用して，各データ間のユークリッド距離を計算し，行列 D に格納する。

  (2)　クラスター間の最小距離を格納する変数 min_dist と，統合するクラスターのペアを格納する変数 to_merge を，それぞれ float('inf'), (0, 1)の値で初期化する。

  (3)　すべてのクラスターを比較して，最も距離が近いクラスター同士を特定する。このとき，変数 i と変数 j にクラスターを格納し，総当たりで計算する。各クラ

スターに含まれるデータの点の座標を X から取り出し，mean 関数を用いてそれぞれのクラスターの代表点を求める。次に，np.linalg.norm 関数を使って，各クラスターの代表点間のユークリッド距離を計算し，変数 dist に格納する。dist が min_dist よりも小さいとき，min_dist の値を dist に，to_merge の値を(i, j)に置き換える。

(4) extend メソッドを用いて，to_merge に格納されたクラスターを統合し，クラスターの数を減らす。

(5) クラスターの数が k より大きい間，(2)～(4)の処理を繰り返す。

(6) 各データがどのクラスターに属するかを記録する配列 labels を作成する処理を行う。各データ点に，どのクラスターに属しているかを示すラベル（ここでは cluster_id）を割り当て，配列 labels に格納する。

• plot_clusters メソッド

クラスター分析の結果をグラフとして表示する。

・処理手順

(1) グラフの大きさを設定する。

(2) クラスターごとにグラフを描画する。scatter 関数は，呼び出すごとに異なる色で点が描画される。

(3) X 軸や Y 軸のラベル，グラフのタイトルや凡例を作成し，グラフを表示する。

ClusterAnalysis クラスの仕様は，次のとおりである。

ClusterAnalysis クラス	
k	クラスターの目標値
labels	各データ点が属するクラスターのラベル
__init__ メソッド	インスタンスの初期化を行うメソッドである。k はデフォルト引数であり，指定がなければ 3 が設定される。
fit メソッド	データのクラスター分析を実行するメソッドである。
plot_clusters メソッド	クラスター分析の結果をグラフで表示するメソッドである。

このプログラムで使用するライブラリ，モジュールおよび関数の説明を，次に示す。

numpy ライブラリ
linalg.norm 関数
linalg.norm(x, 省略) 引数に指定したベクトル，もしくは行列の 2 点間のユークリッド距離を求める。 引　数：本問では，以下の引数を使用する。 x　x ベクトル，もしくは行列 戻り値：2 点間のユークリッド距離

zeros 関数
zeros(shape, dtype, 省略) 指定された形状の配列を作成し，すべての要素を 0 で初期化する。 引　数：本問では，以下の引数を使用する。 shape　配列の形状 dtype　データ型 戻り値：すべての要素が 0 の NumPy 配列
mean 関数
mean(a, 省略) 指定されたデータの平均値を計算する。 引　数：本問では，以下の引数を使用する。 a　平均値を計算するデータ 戻り値：平均値

pandas ライブラリ
read_csv 関数
read_csv(filepath_or_buffer, 省略) CSV ファイルを読み込み，データフレームを作成する。 引　数：本問では，以下の引数を使用する。 filepath_or_buffer　読み込む CSV ファイルのパス 戻り値：データフレーム
DataFrame クラス
二次元の表形式のデータを表す。特定の列のデータを取り出す場合は，「DataFrame クラスのインスタンス['列名']」のように指定する。取り出した特定の列のデータは，リストと同じようにして繰返し処理などに使用できる。引数がない場合，空の DataFrame クラスのオブジェクトを返す。 引　数：本問では，以下の引数を使用する。 data 表形式にするデータ 戻り値：表形式のデータ
values プロパティ
DataFrame.values データフレームのすべての値を NumPy 配列として取得する。 戻り値：NumPy 配列

scipy ライブラリ
spatial モジュール
distance_matrix 関数
distance_matrix(X, X, 省略) 指定されたデータ間の距離行列を計算する。

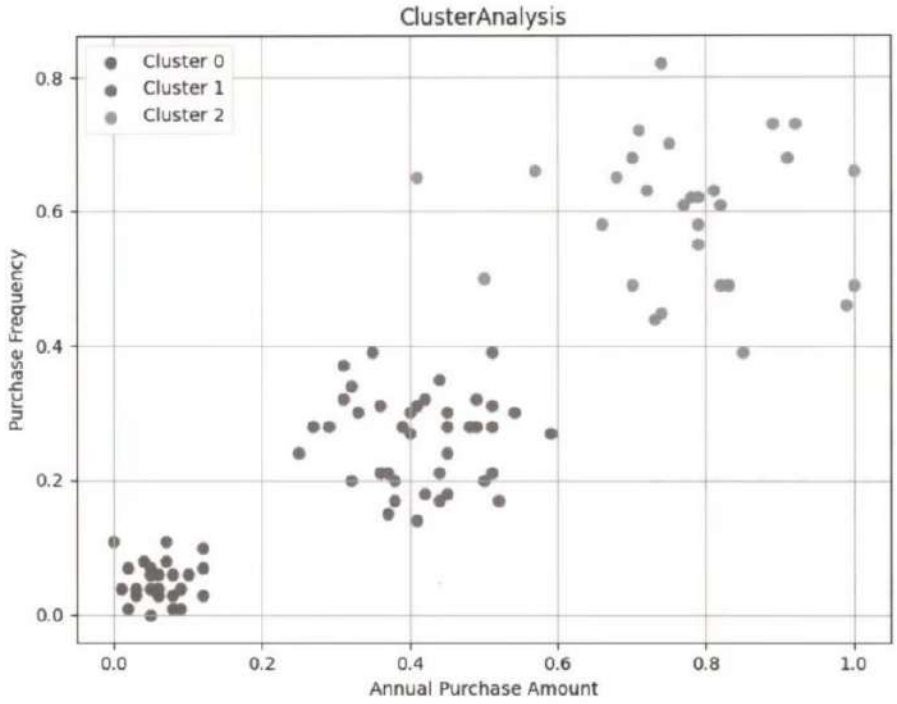
引　数：本問では，以下の引数を使用する。 　　X　クラスタリング対象のデータ 戻り値：ユークリッド距離の行列

matplotlib ライブラリ
pyplot モジュール
figure 関数
figure(figsize, 省略) 指定されたパラメータを基に，グラフを表示する領域を生成する。 引　数：本問では，以下の引数を使用する。 　　figsize　表示領域の横と縦の長さ 戻り値：グラフを表示する領域
scatter 関数
scatter(x, y, label, 省略) 指定されたデータを基に散布図を生成する。 引　数：本問では，以下の引数を使用する。 　　x, y　散布図として描画する対象のデータ 　　label　散布図の凡例ラベル 戻り値：散布図
xlabel 関数
xlabel(xlabel, 省略) X 軸のラベルを設定する。 引　数：本問では，以下の引数を使用する。 　　xlabel　X 軸のラベル
ylabel 関数
ylabel(ylabel, 省略) Y 軸のラベルを設定する。 引　数：本問では，以下の引数を使用する。 　　ylabel　Y 軸のラベル
title 関数
title(label, 省略) グラフのタイトルを設定する。 引　数：本問では，以下の引数を使用する。 　　label　グラフのタイトル 戻り値：グラフのタイトル
legend 関数
legend(省略) 凡例を表示する。

引　数：省略（本プログラムではデフォルト値を使用）
grid 関数
grid(省略) グラフにグリッドを表示する。 引　数：省略（本プログラムではデフォルト値を使用）
show 関数
show(省略) 指定されたデータを基にグラフを作成する。 引　数：省略（本プログラムでは引数を使用せず，デフォルト値を使用）

≪実行結果例≫

下図において，左下付近のデータが Cluster 0，中央付近のデータが Cluster 1，右上付近のデータが Cluster 2 を示す。



<プログラム>

```
1   import numpy as np
2   import pandas as pd
3   import matplotlib.pyplot as plt
4   [ (35) ]
5
6   class ClusterAnalysis:
7       def __init__(self, k=3):
8           [ (36) ]
9           self.labels = None
10
11      def fit(self, X):
12          clusters = [[i] for i in range(len(X))]
13          D = distance_matrix(X, X)
14
15          while [ (37) ]:
16              min_dist, to_merge = float('inf'), (0, 1)
17              for i in range(len(clusters)):
18                  for j in range(i + 1, len(clusters)):
19                      center_i = X[clusters[i]].mean(axis=0)
20                      center_j = X[clusters[j]].mean(axis=0)
21                      dist = np.linalg.norm(center_i - center_j)
22                      if dist < min_dist:
23                          [ (38) ]
24
25              clusters[to_merge[0]].extend(clusters[to_merge[1]])
26              del clusters[to_merge[1]]
27
28          self.labels = np.zeros(len(X), dtype=int)
29          for cluster_id, cluster in enumerate(clusters):
30              for point in cluster:
31                  self.labels[point] = cluster_id
32          return self.labels
33
34      def plot_clusters(self, X):
35          plt.figure(figsize=(8, 6))
36          [ (39) ]:
37              cluster_data = X[self.labels == cluster_id]
38              plt.scatter(cluster_data[:, 0],
39                          cluster_data[:, 1],
40                          label=f'Cluster {cluster_id}')
41          plt.title('ClusterAnalysis')
42          plt.xlabel('Annual Purchase Amount')
43          plt.ylabel('Purchase Frequency')
44          plt.legend()
45          plt.grid()
```

```
46            plt.show()
47
48    data = pd.read_csv('customer_data.csv')
49    X = data[['年間購入金額', '購入頻度']].values
50
51    clustering = ClusterAnalysis()
52    data['Cluster'] = clustering.fit(X)
53    clustering.plot_clusters(X)
```

(35) の解答群

ア from distance_matrix
イ from scipy.spatial import distance_matrix
ウ import distance_matrix from scipy.spatial
エ import scipy

(36) の解答群

ア k = k
イ k = None
ウ self.K = K
エ self.k = None

(37) の解答群

ア clusters < k
イ clusters > k
ウ len(clusters) < self.k
エ len(clusters) > self.k

(38) の解答群

ア dist, to_merge = min_dist, (0, 1)
イ dist, to_merge = min_dist, (i, j)
ウ min_dist, to_merge = dist, (0, 1)
エ min_dist, to_merge = dist, (i, j)

(39) の解答群

ア for cluster_data in range(self.k)
イ for cluster_id in range(self.k)
ウ while range(self.k) < 3
エ while range(self.k) > 3